Mini SOC Log Analyzer - Project Report

A Python tool for SSH authentication log triage

Author: Arman (MCA, Sanjeevani University) | GitHub: mustbearman | Version 1.0.0 | October 2026

1. Overview

The Mini SOC Log Analyzer is a command-line tool written in Python that reads SSH authentication logs, detects repeated failed logins, flags events for analyst review and generates a Markdown incident triage report. It was built as a learning project to practise the first steps of a Security Operations Centre (SOC) Level 1 analyst's work: reading logs, spotting suspicious patterns, and writing up findings for review.

The tool uses only the Python standard library. It was developed and tested on synthetic logs that use documentation-only IP addresses. It is not a SIEM, does not block traffic and does not use machine learning.

Important: every alert is an investigation lead, not proof of an attack.

2. Objectives

1. Parse OpenSSH syslog-style authentication logs reliably and safely.
2. Detect repeated failed logins using a configurable threshold and time window.
3. Highlight the more serious pattern of a successful login following repeated failures.
4. Produce a readable triage report with evidence (log line numbers) and suggested next steps.
5. Handle errors gracefully (missing, empty or wrongly formatted files; invalid arguments).
6. Keep the code small enough for a beginner to read, test and explain.
7. Document limitations honestly.

3. Architecture

The tool is a one-way pipeline of five small modules.

Module	Responsibility
`models.py`	Data containers: `AuthEvent`, `Alert`, `Summary`, `AnalysisConfig`, `ParseResult`
`parser.py`	Reads lines, recognises `Failed ...` and `Accepted ...` events, validates IP address and date
`detector.py`	Applies the two detection rules and calculates summary statistics
`reporter.py`	Builds the Markdown report and escapes untrusted text from logs
`cli.py`	Command-line arguments, terminal summary, exit codes

Data flow: log file -> parser (AuthEvent list) -> detector (Alert list) and summary -> reporter (Markdown) -> report file.

Design choices: standard library only; dataclasses for data; unknown lines are skipped and counted rather than causing a crash; only `Failed ...` lines count as failures so one attempt is never counted twice; alerts never change the exit code.

4. Methodology

4.1 Parsing

Two regular expressions extract the timestamp, host, username, source IP, port and method from `Failed ...` and `Accepted ...` lines. The IP address is validated with Python's `ipaddress` module and the date is validated by building a `datetime`. Syslog lines have no year, so the user supplies one with `--year`. Lines that do not match are counted as skipped.

4.2 Detection rules

Rule 1 - repeated-failed-logins. Failures are grouped by source IP and sorted by time. A two-pointer sliding window checks whether at least *threshold* failures (default 5) fall within *window* minutes (default 5). Overlapping windows merge so one burst gives one alert. Severity is MEDIUM, or HIGH when the count is at least three times the threshold.

Rule 2 - success-after-failures. For each successful login, the tool looks back *window* minutes for failures from the same IP. If at least *threshold* are found, a HIGH alert is raised.

Severity labels are heuristics chosen for this project, not an industry standard.

4.3 Reporting

The report contains a disclaimer, settings, summary numbers, an alert table, per-alert details (evidence line numbers, usernames, suggested next steps), the top failing IPs and a limitations section. Values from logs are placed in code spans or escaped, because usernames are attacker-controlled.

4.4 Test data

`scripts/generate_sample_log.py` creates a synthetic 183-line log with a fixed random seed. IP addresses come from the RFC 5737 documentation ranges. The log contains normal logins, harmless typos, one non-log line and six designed scenarios:

Scenario	Source	Expected result at defaults
Fast brute force, 24 failures in under 4 minutes	203.0.113.45	HIGH alert
Password spraying, 7 usernames	203.0.113.99	MEDIUM alert
6 guesses against one account	198.51.100.77	MEDIUM alert
7 failures then a successful login	192.0.2.88	MEDIUM and HIGH alerts
Near miss, 3 failures	198.51.100.23	No alert
Slow guessing, 5 failures 90 s apart	203.0.113.45	No alert (known blind spot)

5. Setup and usage

Requirements: Python 3.9 or newer. No packages to install.

```
git clone https://github.com/mustbearman/mini-soc-log-analyzer.git
cd mini-soc-log-analyzer
python -m soc_analyzer sample_logs/auth_sample.log --year 2026
python -m unittest discover -v
```

Options: `-o` output file, `-t` threshold (2-10000), `-w` window in minutes (1-1440), `-y` year. Exit codes: 0 finished, 1 input/output problem, 2 invalid arguments.

6. Test results

Tests were run with `python -m unittest discover -v` using Python 3.12.3. Full output is in `docs/test_results.txt`.

Test module	Tests	Covers
`test_parser.py`	14	valid lines, invalid users, bad IP, impossible date, IPv6, binary bytes, missing file and directory

Test module	Tests	Covers
`test_detector.py`	21	threshold edge cases, window limits, merging, severity, success-after-failures, summary
`test_reporter.py`	9	sections, disclaimer, no-alert wording, escaping of hostile usernames, truncation
`test_cli.py`	9	end-to-end on the sample log, threshold/window options, error handling and exit codes
Total	**53**	**Result: OK (all passed)**

One issue found during development: the first version of the report displayed the file name as `auth\_sample.log` because backslash escaping is wrong inside Markdown code spans. It was fixed by adding `code_span()`, and the alert description was also routed through `md_escape()` because the login method comes from the log. Tests were added for both.

Limits of this testing: it uses synthetic data and unit tests only. It has not been tried on large real-world logs, and no performance testing was done.

7. Sample output

Real terminal output from `python -m soc_analyzer sample_logs/auth_sample.log -y 2026`:

```
Input        : auth_sample.log
Lines read   : 183  (events: 85, skipped: 98)
Logins       : 55 failed, 30 successful, 9 unique IPs
Rule         : >= 5 failed logins within 5 min
Alerts       : 5  (HIGH 2, MEDIUM 3)

  ID        SEVERITY  RULE                    SOURCE IP         COUNT
  ALERT-001 HIGH      repeated-failed-logins  203.0.113.45         24
  ALERT-002 HIGH      success-after-failures  192.0.2.88            7
  ALERT-003 MEDIUM    repeated-failed-logins  203.0.113.99          7
  ALERT-004 MEDIUM    repeated-failed-logins  198.51.100.77         6
  ALERT-005 MEDIUM    repeated-failed-logins  192.0.2.88            7
```

The full Markdown report is in `sample_output/sample_report.md`. No screenshots are included; the output above is captured text from a real run.

Reading the result: 203.0.113.45 made 29 failed attempts in total, but only 24 are in the alert because 5 slower attempts the next day fall outside the 5-minute window. Running with `-w 10` or `-t 3` changes which leads appear, which demonstrates the effect of tuning.

8. Limitations

- Counting is per source IP; distributed attacks can stay below the threshold.
- A fixed window misses slow guessing (shown in the sample data).
- Fixed thresholds trade false positives against false negatives.
- Only OpenSSH `Failed` and `Accepted` lines are analysed.
- Logs have no year and logs crossing New Year are not handled.
- No IP enrichment or allow-list; severity is heuristic.
- Offline analysis of one file; no live monitoring.
- Validated on synthetic data only.

9. Future scope

Allow-list for known-good IPs; more log formats (journald, ISO timestamps); year rollover; distributed-attack detection; JSON/CSV export; `--fail-on-alerts`; offline IP enrichment; packaging with `pyproject.toml`; automated tests in GitHub Actions; performance testing on large logs.

10. Conclusion

The project delivers a small, tested and documented log triage tool that can be explained line by line. Its value is in demonstrating log parsing, simple detection logic, safe handling of untrusted input and honest reporting of limitations. It is a learning project and not a replacement for production security monitoring.

Appendix A - Repository contents

`soc_analyzer/` (5 modules, about 665 lines including comments), `tests/` (4 test files, about 404 lines), `sample_logs/auth_sample.log`, `sample_output/sample_report.md`, `scripts/generate_sample_log.py`, `docs/` (architecture, learning guide, limitations, study plan, interview prep, GitHub steps, this report, test results, console output), `README.md`, `LICENSE`, `.gitignore`, `requirements.txt`.